

FROBOZZ MAGIC

Operations Planner

Constraint-Based Production Optimization — Documentation & User Guide

Engine Frobozz OP 1.py · API Frobozz GraphAPI.py · Viewer graph_viewer.html

Source of record: Oracle JD Edwards via Snowflake · Read-only (never writes to JDE)

[At a Glance](#) · [Overview](#) · [Data](#) · [Solver](#) · [Views](#) · [Health Checks](#) · [Cheat Sheet](#) · [Math Appendix](#)

Contents

1. Executive Summary — At a Glance
2. Overview & Architecture
3. Data Sources & Ingestion
4. How the Solver Works (plain English)
5. Outputs, Graphs & Views
6. Business Value
7. Data Health Checks
8. User Guide
9. Key Concepts — One-Page Cheat Sheet
10. Appendix A — The Mathematics
11. Appendix B — Glossary
12. Appendix C — Design Rationale (why each choice)
13. Appendix D — The Methods, in Plain Language

1. Executive Summary — At a Glance

The Frobozz Magic Operations Planner (“OP”) answers one question with rigor: **given today’s work orders, sales orders, inventory, incoming POs and bills of material in JD Edwards, how much customer demand can we actually produce — and what single ingredient or intermediate is blocking each shortfall?**

Inputs	Engine	Outputs
Sales orders, work orders, purchase orders, on-hand inventory (by plant & lot), bills of material, costs, prices, lead times — pulled live from JD Edwards.	Constraint optimization that respects material balance, batch sizes, plant boundaries, lot expiration and time (when supply actually arrives).	Per-order fill rates, the binding constraint behind every shortage, full pegging (what fills what, from which lot), an interactive flow graph, and the financial running total (fill %, shortage \$, margin).

Why it is not just MRP

Classic MRP	Frobozz OP
Explodes demand and tells you what to <i>order</i> , assuming infinite plan feasibility.	Tells you what you can actually <i>make right now</i> with the stock and orders that exist, and <i>why</i> you fall short.
Treats each item independently.	Resolves competition for shared scarce ingredients across many finished goods.
No notion of which shortage matters most.	Computes the marginal value (shadow price) of each constraint and steers scarce supply to the highest-value fills.
Plant/lot/expiry often abstracted away.	Sources by plant, respects lot expiration (FEFO), and honors when POs/WOs actually land.

It is a decision-support and what-if tool. It reads JDE; it never writes back. Planners use it to triage shortages, find bottlenecks, and test changes before committing them in JDE.

“Why can’t I make this order?” — the core idea in one example

Every feasibility answer in this tool comes down to the scarcest input. Concretely:

Worked example — a 1,000-case order

Component	Needed for 1,000 cases	On hand	Supports
Sauce	1,000 lb (1.0 / case)	1,200 lb	1,200 cases
Cheese	500 lb (0.5 / case)	300 lb	600 cases

Feasible production = 600 cases. Binding constraint = Cheese. The order can’t be fully made — not because of sauce, but because cheese runs out first. The planner’s job is then: get more cheese, or accept the 600-case fill. OP computes exactly this for every work order, through every level of the recipe, and tells you the binding item.

This is **Liebig's Law of the Minimum** (§4), and it is the heart of the engine. Everything else — shadow prices, pegging, the graph colors — exists to scale this answer to thousands of orders sharing the same scarce ingredients.

Typical planner workflow

- 1 **Run the planner** against the current JDE snapshot (the live order book).
- 2 **Review the red nodes** — the work orders and finished goods that can't be fully made.
- 3 **Identify the binding constraint** for each shortage (the component at its minimum) from the node, the pegging, or the BOM-chain view.
- 4 **Check stranded inventory** — stock sitting at the wrong plant (a transfer, not a purchase) shows red in On-Hand-by-Plant.
- 5 **Run what-if scenarios** — dial a WO up/down or add a PO and re-solve to see the fill and margin impact before acting.
- 6 **Commit the chosen changes in JDE** — OP is advisory; the planner makes the actual edits in the system of record.

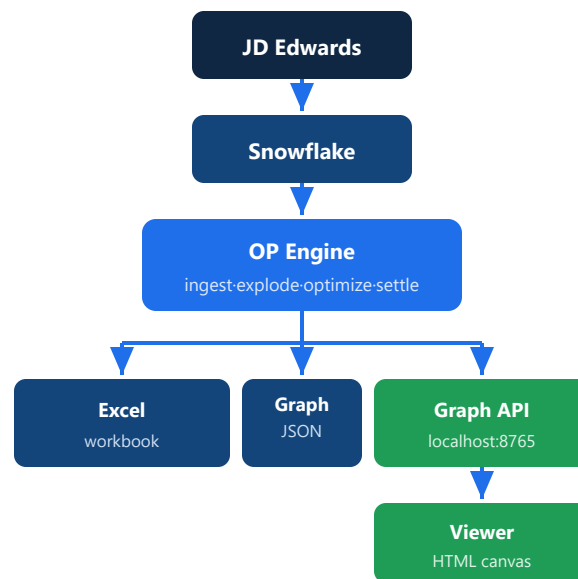
2. Overview & Architecture

Manufacturing feasibility is a **flow-on-a-network** problem governed by two physical laws the planner enforces strictly:

- **Liebig's Law of the Minimum** — a batch is limited by its scarcest ingredient (the cheese, above).
- **Kirchhoff conservation** — material is neither created nor destroyed; what leaves a pool equals what enters its consumers, and a node can never be greener than the worst edge feeding it.

On top of those it layers the realities that make planning hard: shared scarce ingredients fought over by several products, whole-batch (indivisible) production, multi-level WIP and phantom assemblies, separate plants that can't freely share stock, lots that expire, and the calendar (a PO that arrives after a WO starts is no help to it).

System architecture



Graph JSON is the single source of truth the API + viewer read.

Component	File	Role
OP Engine	Frobozz OP 1.py	The brain (~21k lines): connects to Snowflake, runs ~22 queries, explodes BOMs, builds supply pools, runs the solver, settles the plan, writes Excel + graph JSON.
Graph JSON	optimizer_graph.json	The solved plan as nodes + edges (written to the Desktop) — the source of truth the viewer reads.
Graph API	Frobozz GraphAPI.py	Flask server (port 8765): serves the viewer, answers chain/pegging queries, runs what-if scenarios.
Viewer	graph_viewer.html	Interactive canvas of the colored supply network with node panels, pegging, on-hand-by-plant.

End-to-end data flow

1. **Ingest** — ~22 SQL queries pull the live order book into DataFrames.
2. **Explode** — each finished good is expanded FG → WIP → raw/packaging, converting every quantity to a per-unit requirement.
3. **Pool** — available supply per component is assembled: on-hand (by plant/lot) + incoming POs + in-flight WO output.
4. **Optimize** — the three-stage solver finds the production plan that maximizes the chosen objective subject to all constraints.
5. **Settle & peg** — round to whole batches; tag each draw to specific lots (FEFO) for the pegging report.
6. **Publish** — write feasibility, flows and economics to Excel + graph JSON; the viewer renders them.

3. Data Sources & Ingestion

Every number is traceable to a JD Edwards field. The engine reads the same tables JDE's own MRP uses; it never invents data.

Query	JDE table	Provides	Key fields
Sales Orders	F4211	Open customer demand	SDLITM, SDSOQS, SDPPDJ, SDMCU
FG On-Hand	F41021	Finished-good stock (fills SOs directly)	LIPQOH, LIGLPT='FG%'
Component On-Hand	F41021 (+F4101/F4102, F4108)	Component/raw/WIP on-hand by plant & lot, with expiry	LIPQOH, LIMCU, LILOTN, IOMMEJ
BOM	F3002	Parent→component structure & qty/batch	IXKITL, IXLITM, IXQNTY, IXCMCU
Component PO	F4311	Incoming purchase orders by plant	PDLITM, PDUORG, PDPDDJ, PDMCU
Work Orders	F4801	In-flight production & its output	WALITM, WAUORG, WASTRT/WADPL, WAMCU
Forecast	F3460	Demand forecast (below firm SOs)	MFFQT, MFDRQJ
Batch / BA	F4102 / F41002	Batch qty & units-per-production-batch	IBBQTY, UMCONV(BA)
Costs / Prices	F30026 / F4211 / F4106	Standard cost & selling price	IESTDC, SDUPRC, BPUPRC
Lot expiry	F4108	Per-lot expiration date	IOITM, IOLOTN, IOMCU, IOMMEJ
Phantom / Safety / Division	F4102 / F4101	Phantom flags, safety stock, division	IBSTKT, IBSAFE, IMPRP2

How raw values become numbers

JDE stores quantities as scaled integers and dates as Julian `CYYDDD`; the queries de-scale and decode at the source:

Concept	Field	Transform
On-hand / order / PO / WO qty	LIPQOH, SDSOQS, PDUORG, WAUORG	$\div 100$
Standard cost / selling price	IESTDC, SDUPRC, BPUPRC	$\div 10000$
BOM component qty	IXQNTY	$\div 10000 \times \text{UOM chain}$
Batch UOM factor	UMCONV	$\div 10\ 000\ 000$
Dates	*PDDJ, *DRQJ, IOMMEJ	Julian: $\text{year}=1900+[\text{J}/1000]$, $\text{day}=\text{J} \bmod 1000$

Item identity uses the JDE *2nd item number* (long, e.g. `IMLITM` / `IBLITM`); the *short* numeric id (`IMITM`, and F41021's `LIITM`) is the key used to join location records to the item master. Plant codes are normalized (trim, drop trailing `.0`, strip leading zeros) so the same

plant matches across tables.

Plant / lot / expiration model

Component on-hand is a set of lots keyed by **(item, plant, lot, expiration, quantity)**, which enforces three rules:

- **Plant sourcing** — a WO consumes a component only from the plant its BOM sources it from; stock elsewhere is *stranded* (visible but unusable). SOs are exempt (FG ships from any plant).
- **Expiration** — no WO/SO draws a lot expired on its run day; expired lots are removed before optimization and on their expiry day in the timeline.
- **FEFO** — earliest-expiring lot drawn first.

POs are likewise plant-linked: a receipt increases inventory at its branch/plant (`PDMCU`) only, so a PO at a plant a WO can't source from doesn't inflate that WO's usable supply.

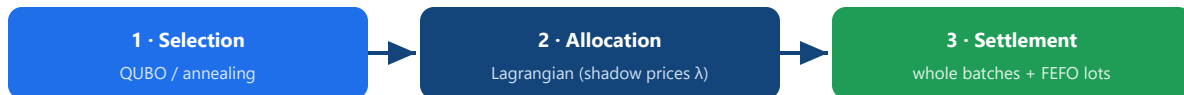
Warehouse / DC supplements. Some stock physically lives in a shared distribution center that no BOM line names as a sourcing plant, so it would read as *stranded* even though production legitimately draws from it. A configurable **plant-supplement** rule fixes this systematically: a component whose BOM sources it from a production plant becomes *also* eligible to draw the warehouse(s) feeding that plant — shipped as 9110→9500 and 8110, 8120→9650 (overridable via `FROBOZZ_PLANT_SUPPLEMENT`). The supplemented lots are the *same* shared FEFO ledger entries, drawn earliest-expiry-first and decremented globally, so two work orders never pull the same case twice. This both clears the false stranding and un-starves production that was waiting on warehouse stock.

Offline test mode

`OFFLINE=1` swaps Snowflake for a fabricated, internally-consistent universe that exercises every problem type (scarce shared ingredients, ghost/phantom WIP, late POs, multi-plant stranding, lot expiry, overage cuts). A known-good run scores **20.00 / 20.00** coverage. It recognizes each query by its output aliases and returns a pre-built sheet — so it validates the Python pipeline but **cannot** validate SQL changes (those must be tested live).

4. How the Solver Works (plain English)

Choosing which products win scarce shared ingredients, in whole batches, to maximize a business objective, is a large combinatorial-plus-continuous problem. The engine solves it in three deliberate stages. (The equations are in Appendix A; this section is the intuition.)



Stage 1 — Selection: which to produce

A simulated-annealing pass explores the combinatorial choice of which finished goods to favor when they compete for the same ingredients, breaking ties and giving the next stage a sensible starting point.

Stage 2 — Allocation: how much, over time (the shadow prices)

The engine searches for the production trajectory that maximizes the objective while honoring material balance over time. Out of this comes λ (**lambda**), a **shadow price** for each ingredient at each point in time: the marginal value of one more unit of that ingredient. λ is *earned from the data*, not tuned — a high λ literally means “this is the constraint costing you fills.” It is time-aware: an ingredient short *before* a big WO runs accrues more pressure than one short only at the horizon’s end.

Stage 3 — Settlement: whole batches, real lots

Production runs in whole batches, so a scarce shared pool is divided among competing WOs **in proportion to their λ** (largest-remainder apportionment). A WO with twice the λ gets twice the share — not winner-take-all.

Proportional, not all-or-nothing. Two WOs draw on a 100-unit pool with $\lambda = 6$ and 3. Each target = $\text{available} \times \lambda_i / \Sigma \lambda \rightarrow 67$ and 33. With small batches you get the clean 2:1 split; only when one batch is large relative to the pool does whole-batch indivisibility force more onto one WO. The WO that loses an indivisible last batch is the pink/dashed “competed” edge in the graph.

Kinds of intermediate — why phantom and ghost WIPs exist

A **WIP** (work-in-process) is an intermediate a plant makes on the way to a finished good — a filling, a dough, a coating. Not every WIP behaves the same way in the source system, and the four cases below are a *reflection of how the ERP and the shop floor actually work*, not choices the optimizer invented. The engine must represent each one as its own kind of node so the multi-level recipe stays complete; if a level were missing, feasibility could not propagate up the chain and conservation could not be checked.

Kind	Why it occurs	How OP models it
Regular WIP	A stocked intermediate with a real work order (it appears in the WO file, F4801). The normal case: something is scheduled to make it.	A WIP work-order node whose own feasibility (a Liebig-min of <i>its</i> ingredients) becomes the supply ceiling for the finished good that eats it.
Phantom ("blow-	An assembly step that exists in the recipe but is never stocked and never separately work-ordered — it is	A pass-through placeholder : it carries no stock and needs no WO of its own; the parent’s demand

through", stocking type 0)	made and immediately consumed <i>inside</i> the parent's production run. ERP defines it to group a sub-recipe, then "blows through" it straight to the real components. This is a deliberate master-data setup (a stocking status of 0 at that plant), not a gap.	explodes <i>through</i> it down to the real raws. Modeling it (rather than pre-flattening the BOM) keeps the chain readable — you can still see the filling step — and lets per-plant status decide, per work order, whether a given draw is really a phantom.
On-hand WIP	A stockable intermediate that <i>already exists in inventory</i> . There may be no open WO because there is no need to make more — it is drawn from stock.	Its on-hand becomes a supply node in the chain (drawn from stock, FEFO by lot), not a thing to re-make. A true phantom carries no stock; on-hand is what makes it a stocked WIP instead.
Ghost	A WIP a recipe needs but for which there is <i>no work order and no on-hand</i> — the demand is real, but nothing is scheduled to make it and none is in inventory. This is a genuine planning gap : the finished good was ordered, but the intermediate it depends on has not been work-ordered yet (a planned WP suggestion may exist, but a WP is only a suggestion, not a WO).	A 0% placeholder shown red/dashed with its ingredient edges, so the missing producer is <i>visible</i> as "this needs a work order." It is deliberately not hidden (which would make the finished good look makeable when it isn't) and never fabricated as makeable (which would violate conservation). See §5 for firing one as a what-if.

Phantom vs. ghost in one line. A **phantom** is a step that is *supposed* to have no work order (it's blown through by design) — so it is a normal, expected placeholder. A **ghost** is a step that *should* have a work order but doesn't — so it is a flag that planning is incomplete. Same shape on the graph (an intermediate with no WO of its own), opposite meanings: one is business-as-usual, the other is a to-do. The distinction is decided **per (item, plant)** from the stocking status (Appendix C.6), because the very same item can be a real stocked WIP at one plant and a blow-through phantom at another.

Objective modes

Mode	Maximizes	Who wins scarce supply
fill_rate (default)	Units / orders filled	Most coverage; λ breaks ties
high_value	Revenue	Highest unit selling price first
profit_minded	Gross margin	Highest unit margin (price – cost) first

Lot ledger, FEFO & time-phasing

- **Lot ledger** — settlement assigns quantities; the ledger tags each draw to specific (plant, lot) stock in **FEFO** order (earliest-expiring first), producing the pegging trace. A lot is tagged once and never double-spent.
- **Time-phasing** — a PO adds supply on its arrival day (a late PO is useless to an earlier WO); a work order's planned output is, by default, **spread evenly across its run window**, so a downstream step can draw the portion produced *by its own date* (a smooth partial-by-timing estimate rather than all-or-nothing on the completion day) — a work order whose window ends after the consumer's date can still only contribute the part finished by then; an expired lot's remainder drops out on its expiry day.
- **Timing modes** — the even-spread above is the default and reflects that a work order's date range is a scheduling *window*, not its few hours of actual run time. It can be switched off (output then lands all on the

completion day), and an optional **daily reclamation** mode re-balances competing demand day by day; reclamation ships **off** by default to keep a stable, committed plan. Both are toggles on the control panel.

Interpreting results at a glance

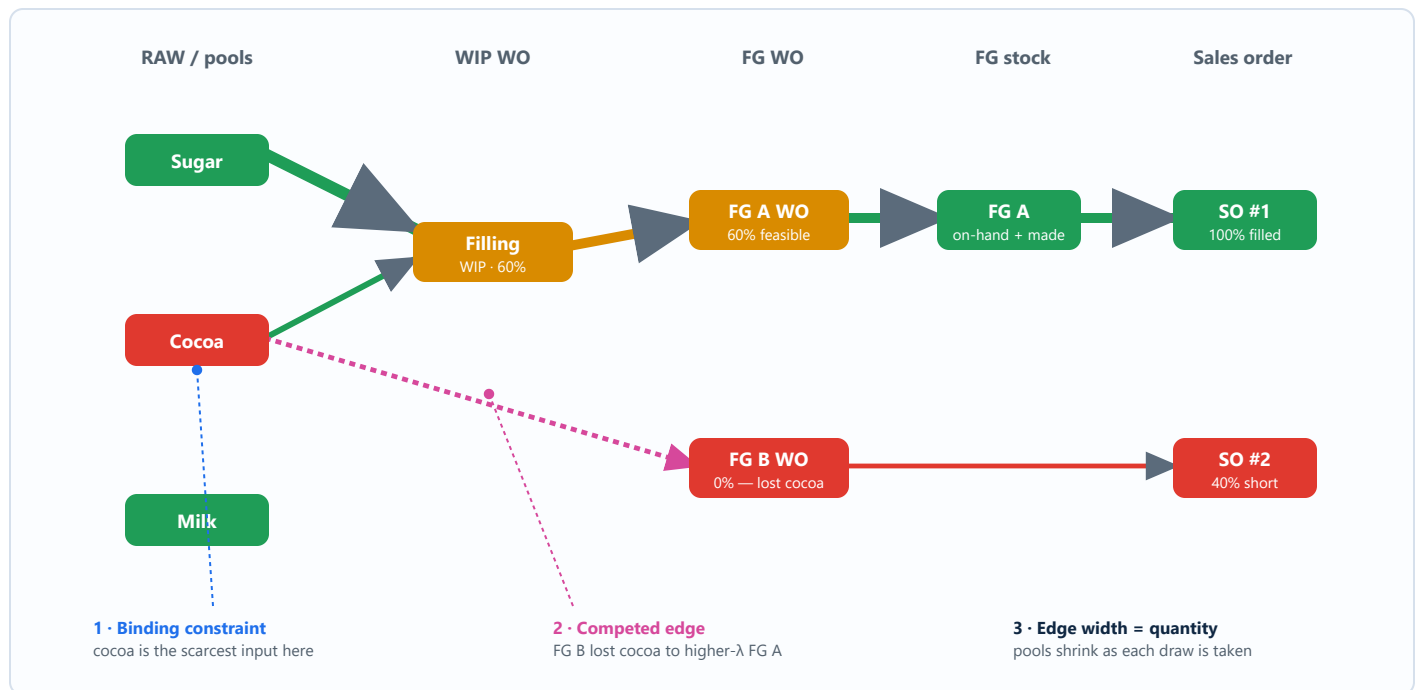
 **GREEN** $\geq 99.95\%$ feasible  **YELLOW** partial  **RED** 0% / no supply  **pink/dashed** competed (lost scarce pool)

- **Red node** → open it or its pegging to find the binding component (the minimum) — that is what to expedite.
- **Pink/dashed edge** → this consumer lost a shared pool to a higher-priority competitor; raising it won't help without more supply or a mode change.
- **Stranded stock** (red in On-Hand-by-Plant) → real inventory at the wrong plant; a transfer candidate, not a purchase.

5. Outputs, Graphs & Views

The flow graph

The solved network is drawn left-to-right: raw/component pools → WIP work orders → finished-good work orders → FG stock → sales orders. Node color = feasibility (above). A solid edge is a real material flow (width = quantity); a **pink/dashed** edge marks a competed loser. The supplying pool's remaining quantity diminishes as each competing edge draws it, so shortfalls are Kirchhoff-visible.



Reading it: cocoa is short, so it is the **binding constraint**. Both the filling WIP (which feeds FG A) and FG B want it; λ -weighted apportionment sends it to the higher-priority path, FG A reaches 60% (its own cocoa-limited ceiling) and ships SO #1 in full from stock + production, while FG B is left at 0 with a **competed** edge and SO #2 only 40% covered from existing FG on-hand. Every colour and width is the solved result, not a sketch.

Node	Meaning
SO	Sales order (demand); status = actual fill vs ordered
FG_WO / WIP_WO	Finished-good / work-in-process work order
RAW	Raw or packaging pool (on-hand by plant + lots)
WIPOH	A stocked intermediate's on-hand, shown as its own supply node — a work-in-process item that already exists in stock is <i>drawn from stock</i> , not re-made. (A true phantom carries no stock; on-hand makes it a stocked WIP, so its on-hand becomes a source node in the chain rather than a number on a phantom.)
FG_STOCK / phantom	FG on-hand; pass-through assembly step

Running total & economics

A top-right panel (and Excel tab) summarizes the plan financially, **before** the plan and **after** any what-if, so the dollar impact of a change is visible at a glance: sales qty/value, short qty/\$, fill % (by qty and value), production cost, and profit margin (filled value – production cost).

The interactive viewer (localhost:8765)

- **Click a node** → feasibility, planned vs feasible, and for a RAW/WIP node the on-hand *by plant* + the FEFO lot list (usable green, stranded red) + the time-phased run-out.
- **Pegging** → what fills each order, when, and from which plant/lot (a WO that can't run pegs nothing).
- **Chain** → walk a finished good down to its binding raw.

The Excel workbook

Tab	Contents
Production Truth	Per-FG planned vs feasible, coverage score, binding constraint per shortage
WO Pegging	What each WO draws, when, from which plant/lot (on-hand rows per-lot, FEFO)
Component Rollup	Per-component total need vs available across all WOs (its own tab; What-If/Δ columns appear when a scenario is active)
On Hand by Plant	Per (component, plant, lot): qty, expiry, Usable vs "STRANDED (wrong plant)"
SO Fill Source	Per sales order, the fill split from existing FG on-hand vs from new production , with a by-month rollup and grand total (Report 1, below)
Running Total	The economics above, before vs after the what-if
Scenario / What-if	Orphans, SO-fill deltas, pegging changes from a scenario recompute

What-if scenarios

What-ifs are run **from the graph, after the plan is solved** — the planner discovers a constraint, then tests a change against it. Each what-if is a **true recompute** through the same three-stage engine (not a linear estimate), so cascading effects — a raised WO pulling scarce raw from a competitor, a freed component reabsorbed elsewhere — are captured faithfully.

What-if	How to run it	What it does
Cut a work order	Select an FG WO node → Run Cuts with a new quantity	Re-solves with that WO capped. Its components are freed and <i>reverse-explode</i> up the chain (a cut FG frees its phantom WIPs, which free their raws); the freed supply is offered to competing demand and may raise other fills. SO fills are time-phased : an order is re-filled only from stock plus production that completes <i>on or before its ship date</i> , so cutting a work order that fed a near-term order correctly shows that order short even when a later work order exists.
Mark a raw always-available	Select a RAW node shown as a limiting factor → ⊕ Mark	Presumes that component has unlimited supply (e.g. stock you know is really available but reads as 0 / stranded) and re-solves, so you see what <i>would</i> fill if it stopped binding. The node turns electric-blue "presumed available."

Graph traces one cut; the running total stacks them. When you focus a cut, the graph highlights only the nodes that cut moved (its own before→after delta, nothing superfluous), while the running-total panel accumulates *every* accepted what-if so the financial picture is cumulative. Reject a run to remove it from both the graph and the Excel.

✅ **Bake to graph — view the what-if as the current state.** Once a what-if is accepted, this button re-optimizes all cuts together and writes the result *onto the nodes* — their colors and numbers (feasibility, order fill, planned quantity) change as if that were the original plan, so the map shows the post-what-if state instead of the baseline. It is **display-only**: the before/after running total and the Excel are left untouched, so you can compare them against the baked map. “Clear what-ifs,” running a new what-if, or reloading reverts the graph. (It bakes the *true* combined result; the running-total panel still shows the fast linear sum — the Cut Effects report quantifies that gap. Further what-ifs after a bake still measure against the original plan.)

No more auto-cutting. Earlier builds auto-trimmed over-target work orders and pre-filled the cut box with that guess. That layer has been **removed**: the solver itself fills **sales orders first, then safety stock, then forecast, and only last lets a WO run over** — and only if no competing demand wanted that supply first. Any overage is therefore intentional, so the cut box now shows the WO’s real planned quantity and every cut you make is genuine.

Ghost work orders & firing them

A **ghost WIP** is an intermediate a finished good’s recipe *needs* but for which **no work order exists** — the demand is real, the WO is not. A ghost is drawn as a red dashed node and is **0% feasible on purpose**: the reason it can’t be made is the *missing work order*, not a shortage. To keep that distinction honest, a ghost still shows its **ingredient edges** — muted-slate dashed “no WO” edges from each BOM component, carrying that component’s real on-hand — so you can see the supply may well exist; it is simply not being pulled because there is nothing to pull it. (A red supply-short edge and a slate no-WO edge therefore mean two different things: one says “buy more,” the other says “raise a WO.”)

Fire a ghost (what-if). From the ghost’s panel you can “turn it on”: set a **planned quantity** and a **WO set-up date** (pick one of the item’s planned WP orders with one click, or type any date), then 🔥 **Fire WO**. This runs the *real* three-stage engine with that work order injected, so its feasibility is **computed from actual supply on that date** — not guessed. It comes back green/yellow/red like any WO, stays *marked as a ghost that has been turned on* (dashed, “👤 ▶ ON”), and a genuinely un-makeable one still returns 0% with its short ingredient named. Turning it off reverts it to a plain 0% ghost. A real WO must still be raised in JDE to make it permanent — firing is a test, not a commitment.

Why a ghost is never allowed to “produce from nothing.” A work order that shows output with no input edges would violate conservation (Kirchhoff) — material appearing from air. The engine enforces the inverse rule everywhere: a WO exists only because it is in the WO file, and its BOM lists its components, so *every*

producer must show its inputs. If a recipe fails to load, that WO is forced to 0% and named in a [NO-BOM] diagnostic rather than being fabricated as makeable — a data gap is surfaced, never hidden by the display.

Planner notes

Any node has a  **note** box in its panel — a place for a planner to record why an order was cut, what a buyer said about a raw, a transfer already in motion, and so on. Notes are keyed to the node's **stable identity** (a finished-good WO by its real WO number, everything else by item number), *not* by the internal graph id, so a note **survives a re-run**: rebuild the plan tomorrow and the note is still on the same order/item. Every note is also collectable in one place — the toolbar's  **Notes** panel lists them all with a jump-to-node link, so nothing has to be hunted down. Notes persist to a small file beside the data, so they also survive restarting the viewer.

Planner reports (viewer buttons)

Four planner reports answer recurring planner questions. Report 1 is a tab in the main Excel; Reports 2–4 are on-demand buttons in the viewer's sidebar header. Each is a self-contained endpoint in `Frobozz GraphAPI.py` that reads the already-solved graph and streams back an `.xlsx`.

Report 1 · SO Fill Source (tab in the main Excel)

Question: how much of each order is filled from stock we already had versus from new production? **How it is built:** during the run, every sales-order fill is split at source into `from_pqoh` (drawn from finished-good on-hand) and `from_wo` (drawn from production). The report lists each SO's order / on-hand fill / production fill / total / short, then rolls the same split up **by ship month** and to an all-months grand total — so you can see, e.g., that this month's fill leans on inventory while next month's depends on production landing on time.

Report 2 · Bottleneck → what it would enable ("● Bottlenecks" button)

Question: for every bottleneck ingredient, how many more of each finished good could we make if that ingredient weren't short? **How it is built — a multi-level marginal explosion:**

1. For every component, compute what the current plan *consumes* (need × feasibility) and its **slack** = available – consumed. **Bottlenecks** are the procurable **raws** whose total need exceeds available (a made WIP isn't something you "get more of").
2. Build the BOM into a **topological order** so a WIP is evaluated before the FG that eats it.
3. For each bottleneck raw, pretend it is unlimited and walk the order: each work order grows up to its remaining gap, **limited by its other inputs' leftover slack** — plus any extra of a WIP input that its now-boosted upstream yields. The extra a WIP makes flows to the FGs that consume it, so freeing strawberry lifts a strawberry→filling→FG chain, not just the direct eaters. Shared slack is consumed greedily (biggest-gap WO first) so two items never double-claim the same leftover.

The sheet lists, per bottleneck: each FG's current feasible, UORG, **Could Produce if Freed**, the **Additional** cases, the **Next Binding Component** (a second short raw, or the WIP whose own production ran out), and a **Link** flag of `direct` vs `via WIP`. So if cream is also short, a strawberry-and-cream item shows a small / zero gain with cream named as its next binding — the report never over-promises an upper bound.

The report also puts a **dollar figure on the leverage**: per finished good, the **Potential Lost Revenue** the shortage is costing (Additional cases × FG selling price); and per bottleneck, its **unit cost**, the **shortfall cost** to close the gap, the **revenue unlocked**, and the ratio — "every \$1 spent on this bottleneck unlocks \$X of sales" —

with a total row. That turns a list of short ingredients into a ranked spend decision: expedite the raw with the highest revenue-per-dollar first. Component cost is the fully-loaded standard (F30026), so the ratio compares like with like across raws and made intermediates.

Report 3 • FG Run-Out (“🔍 FG Run-Out” button — prompts for a list)

Question: for a watch-list of finished goods, when does each run out, what is blocking it, and what unique ingredient is left stranded behind it? **How it is built:** you paste FG item numbers; for each, the report sorts its sales orders by ship date and walks cumulative demand against (FG on-hand + planned production) to find the **run-out date**; reports the FG’s **binding component**; and for every raw **unique to that FG** shows its supply (on-hand + PO), what this FG’s chain consumes, and the **leftover after run-out** — ingredient that will sit idle once the item it was bought for stops shipping. **Uniqueness is transitive through the BOM:** a raw counts as unique to the FG if the whole set of finished goods that consume it — directly *or* through any WIP intermediate — is just that one FG. So a raw that is unique to a WIP that is itself unique to the FG is correctly attributed to the FG (a **via WIP** flag marks those), while a raw shared with any other FG’s chain is excluded.

Caveats, by design: Run-Out compares on-hand + production against cumulative demand and does *not* model exactly *when* production lands; only purchased raws are listed (a made-to-order WIP has no meaningful “left on hand”, though the chain is still walked through it).

Report 4 • Cut Effects (“📊 Cut Effects” button)

Question: what did each what-if cut do on its own, and how does that compare to all the cuts together? **How it is built:** every accepted cut is already re-solved **in isolation** (its own delta is stored), so **Tab 1** lists each cut’s WO, quantity change, and isolated effect (SO fill Δ in cases and dollars, production-cost Δ , margin Δ , work orders moved, top gaining FG). **Tab 2** then runs **one fresh re-optimization with all cuts applied together** and contrasts three rows: the **sum of the individual cuts** (linear), the **true cumulative**, and the **non-linear interaction** between them.

Why the cumulative is not the sum. Cuts interact: freeing one cut’s supply changes what another cut can reabsorb, so re-optimizing all of them at once gives a different total than adding up each cut measured alone. The interaction row makes that gap explicit — the running-total panel shows the fast linear sum, while this report shows the true combined result.

Report 5 • What-If Log (“📝 What-If Log” button)

Question: what changes have I actually made to this plan? **How it is built:** a running plan accumulates several *kinds* of what-if — work-order cuts, raws marked always-available, and fired ghost work orders — and it is easy to lose track of the full set. The What-If Log re-runs the engine with **every** accepted what-if applied at once and writes a single **What-If Actions** tab that records each change on one line with its **before→after**: a cut (UORG before→after), a force-available (real on-hand→presumed unlimited), a fired ghost (0% / no WO→produced of planned, with the resulting feasibility). The same workbook carries the full FG-production, SO-fill, FG-stock and component before/after tabs, so the audit of *what you did* sits next to the effect of *what it did*. It is the plan’s change record — a review artifact and a hand-off note in one.

6. Business Value

The mathematics exists to drive a few concrete business outcomes:

Capability	Business outcome
------------	------------------

Computes the marginal value (λ) of each constrained ingredient	Tells you <i>which</i> expedite or transfer buys the most fill/revenue — spend effort where it pays.
Reallocates scarce supply by the chosen objective	In a shortage, maximizes fill rate, revenue, or margin — deliberately, not first-come-first-served.
Surfaces stranded inventory	Converts a “buy more” into a cheaper inter-plant transfer.
Quantifies revenue at risk and margin	Turns a shortage list into a prioritized, dollar-ranked action list.
True what-if recompute	Test a WO change before committing it in JDE — no guesswork, no wasted runs.

In one sentence: in a typical shortage, OP identifies the marginal value of the constrained ingredients and reallocates supply to maximize the objective you choose — so the limited material makes the most valuable orders, and you know exactly which constraint to relieve next.

7. Data Health Checks

Many planning failures originate in master data, not the optimizer. The engine prints health diagnostics on every run; check these first when results look wrong.

Diagnostic	What it tells you
[COMP-OH DIAG]	Component on-hand actually loaded: row count, total qty, non-zero rows, rows that lost their item identity, FG rows. The first line to check — <code>rows>0</code> means stock is flowing.
[COMP-OH PROBE]	If on-hand came back empty, queries F41021 directly (no joins) to show where non-FG stock actually sits by plant + GL prefix, flagging plants excluded from the plant list.
[QUERY ERROR] '<name>'	A source query failed (e.g. an invalid column) and returned empty instead of silently zeroing — names the exact query and error.
[PLANT SOURCING] / [PLANT DIAG]	The plant restate always runs (strict sourcing, no override); if codes don't reconcile it warns and prints the BOM-vs-on-hand plant universes and per-item mismatches to fix in SQL.
[PO PLANT]	How many PO lines were stranded at non-sourcing plants (or an abort if PDMCU codes don't align).
[EXPIRATION]	How many expired lots were removed.

Plant sourcing is strict. A component's usable supply is *only* the on-hand (and POs) at the plant(s) its BOM sources from — including any supplemental warehouse/DC allowed to feed that plant (e.g. 9110→9500, 8110/8120→9650). A lot at any other plant is stranded and cannot be used: production tied to one plant never pulls another plant's inventory, even if most of the stock happens to sit there (Canada production can't draw Virginia stock). If applying the rule zeros a large share of stocked components, that is a plant-code mismatch between tables — the run *warns and prints the offending codes to fix in SQL*, but it does not override the rule or keep the wrong-plant stock. Expiration is likewise deterministic (F4108 per lot) and applied unconditionally — a lot with no/unparseable expiry is kept (never-expires), so a decode miss errs toward keeping stock.

8. User Guide

Running it

```
py "Frobozz OP 1.py"           # live (Snowflake)
$env:OFFLINE='1'; py "Frobozz OP 1.py"  # offline regression (must score 20.00/20.00)
py offline_jde_stub.py          # regenerate the offline fixture
```

The packaged `.exe` bundles the engine for planners without Python and prompts each user for their own Snowflake credentials (SSO opens a browser). Output (Excel + `optimizer_graph.json`) is written for the viewer; start GraphAPI and open `http://localhost:8765` to explore.

Configuration

Setting	Effect
OFFLINE=1	Use fabricated test data instead of Snowflake
FROBOZZ_MODE	<code>fill_rate</code> / <code>high_value</code> / <code>profit_minded</code> objective
FROBOZZ_DIVISION	Restrict the run to one division's book (DG alone, or the other three together)
FROBOZZ_PLANT_SUPPLEMENT	Warehouse/DC sourcing supplements, e.g. <code>9110:9500,8110:9650;8120:9650</code> (sensible defaults shipped in)
FROBOZZ_FORCE_AVAILABLE	Comma list of components to presume always-available for the whole run (engine-side twin of the graph button)
FROBOZZ_DAILY_SPREAD	Spread each work order's output evenly across its run window for partial-by-timing availability (on by default ; off → output lands on the completion day)
FROBOZZ_DAILY_RECLAIM	Optional hydraulic daily re-balancing of competing demand (off by default — a stable committed plan)
FROBOZZ_LAG_STEP_DAYS	Route 1 to daily λ (exact, slow). Time resolution of the shadow-price (λ) solve: <code>7</code> = weekly (default , keeps the live solve fast), <code>1</code> = genuinely re-solve at daily resolution (finer, ~several× slower). Must match on both the solver and the settlement side (they read the same value).
FROBOZZ_DAILY_LAMBDA	Route 2 to daily λ (simulated, nearly free). Optional daily λ re-pricing (off by default): keeps the weekly solve but <i>simulates</i> a daily re-run with a per-day Augmented-Lagrangian dual step — repricing λ from each day's supply-vs-demand without re-solving. Use this instead of <code>LAG_STEP_DAYS=1</code> when you want daily sharpness but not the runtime (see Appendix C.8).
Per-(item, plant) stocking	Automatic (no flag) — a stocking status of <code>0 / N</code> applies <i>only at the plant where it is set</i> , from <code>F4102.IBSTKT</code> by branch (see Appendix C).
Plant/lot enforcement	Always on (hard-coded) — plant sourcing + expiration always applied

The control panel

A small launcher (`frobozz_control_panel.py`) lets a planner pick the objective mode and division and tune the solver weight sliders before a run; selecting a mode **snaps** the sliders to that mode's preset so you can see what it

emphasizes. A **Hydraulic timing** section adds two checkboxes — **even-spread** (on by default) and **daily reclamation** (off by default) — so the planner can choose the timing model for the run. The window is **scrollable** (it no longer runs off small screens), with the Run/Cancel bar pinned at the bottom.

Lead-planner lock. Free hand-tuning of the weight sliders and saving new defaults are gated behind a password set by the lead planner. Unlock to override weights manually, or hit  **Save as defaults** to make the current settings the baseline every future run starts from. Supply exceptions are deliberately *not* on this panel — you can't know what to override before the data is run, so force-available and the like live on the graph side as post-run what-ifs.

Troubleshooting

Symptom	Check
Every node red / component on-hand 0	Read [COMP-OH DIAG] and any [QUERY ERROR] — a failed source query (bad column) or a plant/expiry decode is the usual cause.
A whole feature is blank (lead times, committed)	Look for that query's [QUERY ERROR] — an invalid identifier means the replica column name differs.
Offline passes but live is wrong	Expected for SQL changes — offline doesn't execute SQL. Verify live.

9. Key Concepts — One-Page Cheat Sheet

Feasibility

The fraction of a work order that is actually makeable now. Green $\geq 99.95\%$, yellow partial, red 0%.

Binding constraint

The single component at its minimum that caps a WO (the “cheese”). The thing to expedite.

Liebig’s Law

Output = the scarcest input. Ten inputs at 100% and one at 40% → a 40% batch.

Kirchhoff conservation

Flow in = flow out; nothing created; a node can’t be greener than its worst feeding edge.

λ (shadow price)

The earned marginal value of one more unit of a constrained ingredient. High λ = “this is costing you fills.”

Proportional allocation

Scarce shared supply splits in proportion to λ (twice the λ = twice the share), in whole batches.

Pegging

The trace of which supply fills which demand, when, and from which plant/lot.

Stranded inventory

Real stock at a plant a given WO can’t source from — a transfer candidate, shown red by plant.

FEFO

First-Expired-First-Out: the earliest-expiring usable lot is drawn first.

What-if

Dial a WO/PO and re-solve; the running total shows the before→after fill and margin.

Competed (pink/dashed)

A consumer that lost a shared scarce pool to a higher- λ competitor.

Objective mode

fill_rate (units), high_value (revenue), profit_minded (margin).

Warehouse supplement

A BOM’s sourcing plant also draws the DC feeding it (9110→9500, 8110/8120→9650) — same FEFO ledger, no double-pull.

Force-available

Mark a raw as unlimited supply and re-solve to see what it was really blocking; node turns blue “presumed available.”

Bottleneck report

Per short raw, the extra of each FG it would enable — direct and via-WIP — each capped by its next binding.

FG run-out

For a watch-list, the date each FG runs out, its binding component, and the unique ingredient left stranded after.

Ghost WIP

A WIP a recipe needs but that has no work order. 0% because the WO is *off*, not because supply is short — shown with slate “no WO” input edges.

Fire a ghost

Give a ghost a qty + a WO date and re-solve; the OP computes its real feasibility from supply. Stays a ghost until a WO is raised in JDE.

Per-plant stocking

A stocking status of 0/N applies only at the plant it is set — the same item can be really-stocked at one plant and presumed-infinite at another.

Planner note

A note on any node, keyed to the order/item so it survives a re-run, and collected in one recall panel.

Appendix A — The Mathematics

This appendix is for developers and analysts who want the formal model. Planners can stop at §4 — nothing here is needed for day-to-day use.

A.1 BOM explosion & quantity-per-unit

$$qpu = (IXQNTY / 10000) \times UOM_{comp} \div BatchSize_{parent}$$

WHERE THE NUMBERS COME FROM

IXQNTY	F3002.IXQNTY $\div$ 10000 — BOM component qty per parent batch.
UOM _{comp}	F41002.UMCONV $\div$ 10 ⁷ — production→stocking UOM factor.
BatchSize _{parent}	F41002 BA conversion or F4102.IBBQTY $\div$ 100; applied only when IXUM='BA'.

A.2 Supply pools & Liebig's Law

$$Supply_c(t) = OnHand_c + \sum PO_c(\tau \leq t) + \sum WO-output_c(\tau \leq t)$$

$$Feasible(WO) = \min_{c \in BOM} \lfloor Supply_c \div qpu_c \rfloor, \text{ capped at OrderQty}$$

WHERE THE NUMBERS COME FROM

OnHand _c	F41021.LIPQOH $\div$ 100, sourcing plant, unexpired (expiry from F4108).
PO _c	F4311.PDUORG $\div$ 100 at PDPDDJ, plant PDMCU.
WO-output _c	F4801.WAUORG $\div$ 100 at completion WADPL, plant WAMCU.
OrderQty	F4801.WAUORG $\div$ 100.

The component achieving the minimum is the **binding constraint**. For multi-level products the law applies up the chain: a WIP's own feasibility (itself a Liebig-min of its raws) becomes the supply ceiling for the FG that consumes it.

A.3 The three-stage solver

Stage 1 — QUBO selection

A Quadratic Unconstrained Binary Optimization over "favor this FG / run this WO?" decisions, with rewards for filled value and penalties for over-subscribing shared scarce pools, solved by **simulated annealing** to seed Stage 2.

Stage 2 — Augmented Lagrangian relaxation (ALM)

Hard material-balance constraints are relaxed into the objective by the **Augmented Lagrangian Method**: each constraint carries a multiplier λ (its shadow price) *and* a shared quadratic penalty ρ . The engine finds the

continuous production trajectory u minimizing the augmented Lagrangian:

$$L(u; \lambda, \rho) = C(u) + \sum_{r,t} \lambda_{r,t} \cdot g_{r,t}(u) + \sum_w \lambda_w \cdot g_w(u) + (\rho/2) \cdot [\sum_{r,t} g_{r,t}(u)^2 + \sum_w g_w(u)^2]$$

The two constraint terms are the defining pieces of an ALM: a **linear** multiplier term $\lambda \cdot g$ (the true Lagrangian price) plus a **quadratic augmentation** $(\rho/2) \cdot g^2$ that keeps the inner problem well-posed even before λ is earned. This replaces the earlier build's fixed-penalty form (a single constant Λ multiplying $\lambda \cdot g^2$, with no separate linear term).

WHERE THE NUMBERS COME FROM

u	Decision variable (production rate per FG per step); bounded $0 \leq u \leq F4801.WAUORG \div 100$.
$C(u)$	–(filled value): units $\times$ price ($F4211.SDUPRC \div 10000$, fallback $F4106.BPUPRC$); <code>profit_minded</code> subtracts cost ($F30026.IESTDC \div 10000$).
$g_{r,t}$	Normalized binding residual <code>clip(max(0, demand_{r,t}(u) – supply_{r,t}) ÷ capacity_r)</code> — a scale-free fraction; demand from BOM $\times u$ (§A.1), supply from §A.2. The <i>same</i> residual feeds the penalty and the dual update (a matched primal–dual pair).
λ	Shadow prices, earned by the ALM dual update below (start at 0, not from JDE).
ρ	Adaptive penalty weight and dual step size (start 10). It <i>grows</i> ($\rho \leftarrow \min(\rho_{\max}, \gamma \cdot \rho)$) when the constraint violation fails to fall, and <i>holds</i> when it improves; capped so λ and ρ stay bounded on genuinely infeasible (shortage) problems.

Two nested loops solve it: a **primal** step (hold λ and ρ , minimize L over u with bound-constrained L-BFGS-B), then an ALM **dual** step that moves each multiplier by ρ *times* its residual (in ALM the penalty ρ is the dual step — not a separate learning rate), followed by the adaptive penalty update:

$$\lambda_{r,t} \leftarrow \max(0, \lambda_{r,t} + \rho \cdot (\text{demand}_{r,t} - \text{supply}_{r,t}) / \text{capacity}_r) \quad \rho \leftarrow \min(\rho_{\max}, \gamma \cdot \rho) \text{ if violation stalls}$$

The residual is signed: a binding resource's λ rises; a slack resource's relaxes toward 0. At convergence this satisfies **complementary slackness** — $\lambda > 0$ *only* where the resource truly binds — so the converged λ is the genuine time-phased shadow price. On a feasible plan the residual reaches 0, so $\lambda \rightarrow 0$ and ρ never grows; on a shortage plan ρ climbs (capped) and λ keeps ranking the binding constraints. Stage 3 consumes λ only as a *relative* ranking, so the ALM's scale is immaterial downstream.

Stage 3 — settlement & whole-batch apportionment

The continuous trajectory is settled discretely. Each competing WO's target on a scarce pool is $\text{available} \times \lambda_i / \sum \lambda$, taken in whole batches, with leftover batches to the largest fractional remainders (ties to higher λ). A conservation-safe top-off assigns the final sub-batch remainder so total consumed equals available exactly.

A.4 How Kirchhoff and Liebig are preserved in tandem with the λ weighting

The three mechanisms are not alternatives; they operate together, in order, and each guarantees a different property:

Mechanism	What it decides / guarantees
-----------	------------------------------

λ weighting (Stage 2→3)	How a <i>contested</i> pool is <i>split</i> among competing consumers — proportional to earned shadow price.
Liebig (per consumer)	Caps each consumer by its <i>own scarcest</i> input. Winning a large λ -share of ingredient A does not raise a WO above what ingredient B (its binding input) allows.
Kirchhoff (post-pass)	Guarantees the splits <i>sum to $\leq$ the pool</i> and nothing is double-spent; caps each supply edge at the node's proportional need and releases the excess.

The sequence is: (1) the λ -weighted apportionment proposes how much of each scarce pool each WO receives; (2) each WO's output is then clamped to its Liebig minimum across *all* its components — so a WO that won plenty of one ingredient but is starved of another is still limited by the starved one; (3) a conservation pass reconciles the result — it sets each node's displayed feasibility equal to its final flow, caps every incoming supply edge at $\text{need} \times \text{feasibility}$, releases any over-pull back to the pool, and audits that no node is greener than its worst feeding edge.

apportion by $\lambda \rightarrow$ clamp by Liebig (min over inputs) $\rightarrow$ reconcile by Kirchhoff ($\Sigma \text{draws} \leq \text{pool}$)

Because the λ -split only ever *distributes* what exists and the Kirchhoff pass re-caps to actual need, material balance holds to the unit (offline audit target: zero contradictions). In short: **λ decides the split, Liebig caps each consumer, Kirchhoff conserves the whole** — the optimization can prioritize value without ever inventing or double-counting material.

Appendix B — Glossary

BOM — Bill of materials (F3002): the parent→component recipe.

WIP — Work-in-process: an intermediate made and consumed in-plant.

Phantom — A non-stocked pass-through assembly step (stocking type 0).

Ghost — A WIP a recipe needs but with no work order to make it; 0% because the WO is off, not because supply is short.

Fire a ghost — A what-if that injects a WO (qty + set-up date) for a ghost and re-solves so its feasibility is computed from real supply.

No-WO edge — A ghost's slate/dashed ingredient edge: shows what it *would* pull, so a missing WO never reads as a shortage.

Per-(item,plant) stocking — A stocking status (0/N) that applies only at the plant it is set (from F4102.IBSTKT by branch).

Daily λ re-pricing — A per-day dual-step correction to the weekly-solved shadow price, sharpening it near a due date without re-solving.

Liebig's Law — Output limited by the scarcest input (the minimum).

Kirchhoff — Conservation: flow in = flow out; nothing created.

QUBO — Quadratic Unconstrained Binary Optimization (Stage-1 selection).

L-BFGS-B — Bound-constrained quasi-Newton optimizer (Stage-2 primal).

Dual ascent — Iteratively raising the price (λ) of binding constraints.

Augmented Lagrangian (ALM) — Stage-2 method: enforces material balance with a linear multiplier term $\lambda \cdot g$ *plus* an adaptive quadratic penalty ($\rho/2 \cdot g^2$; the dual update $\lambda \leftarrow \max(0, \lambda + \rho \cdot g)$ earns the shadow price.

ρ (rho) — ALM penalty weight & dual step size; grows when the constraint violation stalls, holds when it improves (capped).

λ (lambda) — Shadow price: earned marginal value of a constrained resource.

Complementary slackness — $\lambda > 0$ only where a resource truly binds.

FEFO — First-Expired-First-Out lot draw order.

Pegging — The trace of which supply fills which demand, when, from where.

Stranded — Real stock at a plant a given WO cannot source from.

Largest-remainder — Whole-batch apportionment giving leftover batches to the largest fractional shares.

Julian CYYDDD — JDE date: century-flag, 2-digit year, day-of-year.

field_a — The Stage-3 settlement layer (whole-batch, conservation-exact).

Plant supplement — A rule letting a component also source the warehouse/DC feeding its BOM plant, clearing false stranding.

Force-available — A what-if presuming a component has unlimited supply, to reveal what it was actually blocking.

Slack — A component's available supply minus what the current plan already consumes (the headroom another WO could draw).

Next binding — The second constraint that caps an FG once its first bottleneck is freed (a short raw or a capped WIP).

Run-out date — When an FG's on-hand + production stops covering cumulative sales-order demand.

Appendix C — Design Rationale (why each choice)

Appendix A is *what* the engine computes; this appendix is *why it is built that way* — the blueprint notes. Each entry states the problem, the choice, why that choice over the obvious alternatives, and the trade-off accepted. If §4 is the machine running, this is the reason each part is the part it is.

C.1 Three stages instead of one solver

Problem. The real problem is mixed: a *discrete* choice (which orders win scarce shared ingredients, run in whole batches) wrapped around a *continuous, time-phased* flow (how much of each thing over which days). **Choice.** Split it into **selection (QUBO)** → **allocation (Augmented Lagrangian)** → **settlement (whole-batch, FEFO)**, each stage handing the next a warm start. **Why not one MILP?** A single mixed-integer program over batch counts $\times$ days $\times$ every FG is combinatorially large, slow, and brittle when infeasible — and a shortage plan is *always* infeasible in the strict sense (you cannot fill every order). The three-stage split lets each layer use the tool it is best solved with: annealing for the combinatorial seed, a smooth relaxation to *price* the constraints, and a discrete conservation-exact pass to place whole batches. **Trade-off.** It is not a certificate-of-optimality solver; it is a fast, explainable, always-returns-a-usable-answer solver — the right call for a daily planning tool whose output a human must trust and act on.

C.2 λ is earned from the data, never seeded from JDE

Problem. The whole point is to know *which* constraint is costing fills. **Choice.** Start every shadow price at **zero** and let the Augmented-Lagrangian dual update raise only the ones that truly bind (complementary slackness — §A.3). **Why not seed λ from cost, priority, or a heuristic?** A seeded price bakes in an *assumption* about what matters and then the plan “confirms” it — circular. An earned λ is falsifiable: if a resource never binds, its price stays 0 no matter how expensive it is; if it binds hard, its price climbs even if nobody flagged it. That is what makes “expedite this one” a finding rather than a restatement of the input. **Trade-off.** A few dual iterations of compute, and λ is only a *relative* ranking (its absolute scale is meaningless) — which is exactly why Stage 3 consumes it as a ratio and nothing downstream depends on its units.

C.3 Whole-batch largest-remainder apportionment, not LP rounding

Problem. Production is indivisible — you run a 2,000-lb batch or you don’t — but the λ -split gives fractional targets. **Choice.** Give each competitor $\text{available} \times \lambda_i / \sum \lambda$, floor to whole batches, then hand the leftover batches to the **largest fractional remainders** (ties to higher λ), and top-off the last sub-batch so $\sum \text{draws} = \text{available}$ exactly. **Why not just round the LP solution?** Independent rounding double-spends a shared pool (two WOs each round *up* off the same 100 units) or strands it (both round down) — either way conservation breaks. Largest-remainder is the classic apportionment method precisely because it is conservation-exact by construction and proportional in the aggregate. **Trade-off.** When one batch is large relative to the pool, indivisibility forces the whole batch onto one WO and the other becomes the visible **competed** loser — honest, and drawn as such, rather than smeared into a fractional fiction.

C.4 λ splits, Liebig caps, Kirchhoff conserves — in that order

Problem. Three different things can go wrong: the wrong order wins scarce supply; an order looks makeable on one ingredient while starved of another; or the graph shows more material flowing than exists. **Choice.** Three mechanisms, each owning exactly one property, applied in sequence (§A.4): λ decides the *split* of a contested

pool; Liebig then *clamps* every consumer to its own scarcest input; Kirchhoff finally *reconciles* so draws never exceed the pool and no node is greener than its worst feeding edge. **Why this order?** Pricing must come first (you cannot cap what you have not allocated); the min-law must come before conservation (a WO that won plenty of A but is starved of B has to fall to B's level *before* you reconcile totals); conservation is last because it is the referee, not a player. **Trade-off.** Three passes instead of one clever objective — but each is independently checkable, and the offline audit target of *zero* conservation contradictions is only reachable because the responsibilities are separated.

C.5 Strict plant sourcing, with a supplement as the only escape hatch

Problem. Inventory sits at many branches; a work order can only really consume what is at (or feeds) *its* plant.

Choice. A component's usable supply is *only* the on-hand/POs at its BOM sourcing plant — a lot anywhere else is **stranded** and shown red — with one explicit exception: a configured warehouse/DC that feeds a plant (e.g. 9110–9500) is allowed in, through the same FEFO ledger. **Why not just pool all stock?** Pooling makes the plan optimistic in a way that fails on the floor: Canada production cannot draw Virginia stock, and a plan that assumes it will quietly under-deliver. Strict sourcing turns that hidden risk into a visible, actionable *transfer candidate*. **Why an explicit supplement rather than fuzzy matching?** Because a real DC relationship is a known fact, not a guess — encoding it as a named rule keeps the “can this plant use this stock?” answer auditable. **Trade-off.** If plant codes disagree between tables, strict sourcing can zero real stock — so the run *warns and prints the offending codes to fix in SQL* rather than silently relaxing the rule. It never hides a data problem by loosening the model.

C.6 Stocking status is per-(item, plant), not per-item

Problem. A stocking status of `0/N` means “don't track this as stocked inventory here” — but *here* is a plant. The same item can be a normally-stocked product at one branch and a blow-through/non-stock at another. **Choice.**

Read the status from `F4102.IBSTKT` keyed by **(item, plant)**, and apply it only where it is set: presume infinite supply for that item *at that plant*, and treat a WIP as a **phantom** (pass-through, no WO) only when it is sourced from a plant where its status is 0 — regular where it is really stocked. **Why not the per-item stocking flag?**

Collapsing to per-item was over-broad: it made an item phantom (or infinite) everywhere because of one branch's setup, which both invented supply where there was none and erased real WOs elsewhere. **The sharpening**

insight: “it is the *work order* that is blow-through, not the item” — so the classification lives on the WO's sourcing plant, not on the item globally. **Trade-off.** When no per-plant record exists the item is treated as normally-stocked (the safe default), and the infinite-supply presumption is aggregate rather than per-consumer — erring toward *more* availability at the flagged plant, the intended direction.

C.7 A work order's output spreads across its window (window ≠ run time)

Problem. A work order's date range is a *scheduling window*, not the few hours it actually runs — yet a downstream step that overlaps only part of that window needs to know how much is ready *by its own date*.

Choice. By default, spread a WO's planned output **evenly across its whole run window** in the lot ledger, so a consumer partway through gets the portion finished by then — a smooth partial-by-timing estimate. Off (a toggle), output lands entirely on the completion day (all-or-nothing). **Why even-spread as the default?** Lumping everything on the completion day makes a legitimately-overlapping consumer read as getting *nothing*, which cascades into a fabricated “no supply” shortage and an over-pull elsewhere — the ledger disagreeing with the feasibility it is supposed to support. Even-spread makes the ledger equal the supply curve the solver used, so timing is a smooth gradient rather than a cliff. **Trade-off.** Even-spread is an approximation of the true (unknown)

intra-window production profile; it is predictive, not descriptive — which is why it is the default but can be switched to the conservative completion-day model when a planner wants the strict view.

C.8 Daily λ re-pricing is a dual-step correction, not a re-solve

Problem. Shadow prices are solved at a coarse (weekly) time step to keep the live run fast — the λ solve is a big share of the ~1,000-second live runtime, and simply shifting its granularity from weekly to daily could multiply that several-fold. Yet scarcity is really a *daily* phenomenon — a raw short the day before a big order is worth more than the week-average implies — so daily prices are what you actually want. **Choice: simulate the daily re-run instead of paying for it.** Solve the Lagrangian once, weekly; then, rather than re-solving the whole thing at daily resolution, **reprice the multiplier day by day** from each day's updated supply-vs-demand. At each work order's actual due-day, take one more Augmented-Lagrangian dual step, $\lambda_{\text{day}} = \max(0, \lambda + \rho \cdot g_{\text{day}})$, where g_{day} is that component's over-subscription tension *by that day* (cumulative demand $\div$ cumulative supply $- 1$) and ρ is the ALM's *own converged* penalty as the step size. **Why this reproduces a daily re-solve.** Adjusting a multiplier from the constraint's current tension is *exactly* what a re-solve's dual iterations do — so one dual step per day moves λ in the same direction a genuine daily solve would, at a fraction of the cost. That is the optimal-control (Pontryagin) insight: the multiplier is a **costate** you can propagate with a local update; you never have to re-solve the primal to *re-price*, only to step the dual along the day's residual. One dual step per day is therefore a first-order (predictive) stand-in for the expensive daily solve. **Why no tuning knob?** An early version added a hand-tuned "gain" on the correction — but the ALM already has a converged step size (ρ); a separate gain just duplicated it. Using ρ keeps a single, principled step and removes a slider that could only be set wrong. **Trade-off.** It is a first-order correction on a *fixed* primal (the production quantities are not re-optimized), and it reprices from the *planned* day-phased supply/demand rather than the realized post-settlement depletion (a further refinement). The two routes to daily λ thus trade off directly: `FROBOZZ_LAG_STEP_DAYS=1` genuinely re-solves at daily resolution (exact, several $\times$ slower); `FROBOZZ_DAILY_LAMBDA` simulates it with this per-day dual step (approximate, nearly free) — which is why the simulated one exists and is the one you reach for when runtime matters.

C.9 What-ifs recompute; a pure cut cascades incrementally

Problem. A planner needs to test a change ("cut this order," "assume this raw is available") and see the *true* ripple, not a linear guess. **Choice.** Run every what-if **after** the plan is solved, through the *same* engine, so cascading effects are real. But treat the two directions differently: adding supply (force-available, firing a ghost) is a forward re-run; a **pure cut** — which can only *free* material — is applied as a monotone **incremental cascade** on the saved baseline rather than a full re-solve. **Why not re-solve a cut too?** A full re-solve re-rounds whole batches between *unrelated* finished goods, so a cut to FG A could nudge FG Z for no causal reason — noise that destroys the clean "this cut did exactly this" trace. The incremental cascade guarantees monotonicity: the cut FG and its chain drop, beneficiaries that share the freed ingredient rise, and nobody else moves — with Kirchhoff still balancing freed = reabsorbed + orphaned. **Trade-off.** The graph shows one cut's isolated trace while the running total stacks all cuts (the fast linear sum); the *true* combined result — which differs because cuts interact — is what the Cut Effects report's cumulative row and the "bake to graph" both compute, so the approximation is always reconcilable to the exact answer.

C.10 The graph is a faithful mirror — the display never "fixes" the data

Problem. When a node looks wrong, the cheap fix is to correct it *in the drawing*. **Choice.** Never do that. Node feasibility comes from the settlement layer; edges come from the actual move-log; the viewer recolors and overlays but does not recompute or invent flow. **Why the hard line?** The guiding rule: *“if the data is wrong and you hide it by correcting the display, that is always the wrong thing to do.”* A prettier graph that disagrees with the plan is worse than an ugly one that agrees — it hides the very defect a planner needs to see (a mismatched plant code, a recipe that didn’t load, a stranded lot). So the fixes go to the *root* (the query, the ledger, the classifier) and the graph is left honest. **Trade-off.** More work than a display patch, and sometimes an uglier picture — but the picture is *true*, which is the only property that makes it worth acting on.

C.11 A ghost still shows its inputs; nothing produces from nothing

Problem. A recipe can need an intermediate for which no work order exists (a **ghost**). The lazy rendering — a floating 0% node with no edges — both looks like “produces from nothing” and hides *why* it is 0%. **Choice.** Draw the ghost’s BOM inputs as **slate “no WO” edges** (distinct from red supply-short edges), carrying each ingredient’s real on-hand. And enforce the inverse invariant on *real* producers: every work order must show its inputs; a recipe that fails to load forces the WO to 0% and a **[NO-BOM]** diagnostic rather than being fabricated as makeable. **Why bother, since a ghost is 0% either way?** Because the *reason* is the whole message. A red edge says “buy more”; a slate no-WO edge says “raise a work order” — opposite actions. Showing supply that *exists but isn’t pulled* is what makes “the WO is turned off, not the cupboard bare” legible at a glance. It is also Kirchhoff hygiene: a producer with no visible inputs violates conservation on sight. **Trade-off.** A few more edges on the graph — deliberately styled so they can never be mistaken for real flow.

C.12 Firing a ghost computes feasibility from the OP, and needs a date

Problem. “What if this ghost had a work order?” is a real planning question, but a made-up answer is worthless. **Choice.** Firing a ghost injects a genuine work order (planned qty + set-up date) and re-runs the engine, so the answer is **computed from actual supply on that date**, not guessed — and it stays *marked as a ghost that has been turned on*, because no real order exists until one is raised in JDE. **Why require a date?** A work order’s timing decides whether its output lands in time for the orders it feeds; without a date the feasibility is meaningless. The planner ideally picks one of the item’s existing planned **WP** suggestions (a real candidate), but may enter any date. **Why keep the ghost mark after firing?** Honesty again — a fired ghost is a *hypothesis*, and letting it masquerade as a committed order would be the same “fix it in the display” sin as C.10. **Trade-off.** Firing is a test that must be followed by a real WO in JDE to become real — the tool makes the consequence visible but does not pretend to have changed the system of record.

C.13 Planner notes are keyed to a stable identity

Problem. A note is worthless if it vanishes on the next run. Internal graph ids are synthetic and change when the plan is rebuilt. **Choice.** Key each note to the node’s **durable identity** — a finished-good WO by its real WO number, everything else by item number — and persist it beside the data. **Why not key by node id (the easy thing)?** Because the node id is an artifact of one build; keying to it would silently orphan every note the next morning. Item/WO numbers are the identifiers a planner actually thinks in and that survive a rebuild, so the note stays attached to the *thing*, not the drawing of it. **Trade-off.** If a note’s item is genuinely gone from a later plan, it is flagged “off-graph” and reachable by search rather than silently dropped — the note is never lost, only marked as needing a home.

C.14 The change log records every kind of what-if

Problem. A worked plan accumulates cuts, force-availables, and fired ghosts; a report that logs only cuts is an incomplete record. **Choice.** The What-If Log's **What-If Actions** tab records *every* change on one line with its before→after, next to the FG/SO/stock/component effect tabs. **Why one unified log?** A plan is only as reviewable as its change record; a partial log invites "where did this number come from?" A single audit of *what you did* beside *what it did* makes the plan hand-offable and defensible. **Trade-off.** One extra recompute to produce it — cheap for an artifact that doubles as a review note and a hand-off.

The thread through all of these. Every decision above resolves the same way: **compute the honest answer and show it, even when a shortcut would look nicer.** Prices are earned not assumed (C.2); material is conserved not smeared (C.3, C.4); stock is sourced where it really is (C.5, C.6); timing is modeled not lumped (C.7, C.8); what-ifs are recomputed not estimated (C.9, C.12); and the display mirrors the data instead of papering over it (C.10, C.11). The tool's value is that a planner can trust what it shows — and trust survives only if the model never lies to make the picture prettier.

Appendix D — The Methods, in Plain Language

Appendices A and C name several techniques — QUBO, simulated annealing, the Lagrangian, L-BFGS-B — and assume you know what they are. This appendix explains each one from scratch: **what it is**, in everyday terms, and **why it is the right tool for its stage**. None of it is needed to *use* the planner; it is here so the engine is not a black box.

D.0 Why the problem needs more than one tool

The planning problem has two very different halves. One is a pile of **yes/no choices** that interact — which orders do we favor when they fight over the same scarce ingredient? — a *combinatorial* problem. The other is a question of **how much, over which days** — a smooth, *continuous*, time-phased flow problem, subject to the hard rule that you can't use more of something than exists. No single algorithm is best at both. So the engine uses a tool suited to each half and passes the result along: a combinatorial search to make the yes/no choices, a continuous optimizer to set the quantities and *price* the constraints, and a discrete settlement to place real whole batches. Appendices below explain each tool in that order.

D.1 QUBO — the language for the yes/no choices

What it is. QUBO stands for **Quadratic Unconstrained Binary Optimization**. “Binary” = every decision is a switch, 0 or 1 (favor this finished good? yes/no). “Quadratic” = the score can reward or penalize *pairs* of switches together, not just each alone — which matters because the interesting effects are about *competition between two orders* for the same pool (a pairwise interaction). “Unconstrained” = there are no separate hard rules; every rule is folded *into* the score as a reward or penalty, so *any* pattern of switches has a number and you just look for the highest. In short, QUBO is a way to turn “which combination of choices is best?” into a single scoring formula over on/off switches.

Why we use it here. The Stage-1 selection problem is a set of interacting on/off choices (favor this order, run that WO), and its key effect — two orders over-subscribing one scarce pool — is naturally a pairwise (quadratic) penalty. QUBO is the standard, compact way to write exactly that, and it can be solved by fast, general heuristics without a heavyweight integer solver. We only need a *sensible* combination to hand to Stage 2, so a scoring-and-searching approach is the right weight of tool.

D.2 Simulated annealing — the search that solves it

What it is. Simulated annealing is a search strategy borrowed from metallurgy. When you cool metal slowly, its atoms settle into a low-energy, well-ordered state; cool it too fast and it locks into a flawed one. The algorithm imitates that: it starts “**hot**” — willing to accept even *worse* moves — so it roams widely and doesn't get trapped early; then it “**cools**” — becoming progressively pickier, accepting only improvements — so it settles into a good solution. The willingness to occasionally go downhill early is exactly what lets it climb *out* of a shallow dead-end to find a deeper one.

Why we use it here. A QUBO score built from competition penalties is a **bumpy landscape** with many local peaks; a greedy “flip whichever switch helps most” search gets stuck on the first small peak it finds. Annealing's hot-then-cool schedule escapes those traps, it is cheap, and it can be stopped anytime with a usable answer — all we need for a Stage-1 seed. We are not chasing a provably perfect selection here; we are getting a good starting point quickly, which is annealing's sweet spot.

D.3 The Lagrangian & shadow prices — pricing the constraints

What it is. Many problems have **hard constraints** — here, “don’t consume more of an ingredient than exists.” A **Lagrangian relaxation** is a classic trick: instead of forbidding a constraint outright, you *remove the wall* and attach a **price** to crossing it, added into the objective. The optimizer is now free to overshoot, but it *pays* for every unit it does; raise the price high enough and it chooses, on its own, not to overshoot. That price is the **Lagrange multiplier**, written λ . The beautiful part: when you tune λ to the point where the constraint is just satisfied, λ equals the constraint’s **shadow price** — the marginal value of relaxing it by one unit, i.e. *how much more objective you’d gain from one more pound of that ingredient*. A resource that isn’t scarce prices out at 0; a binding one prices high. So “solve for λ ” and “find which constraint is costing you the most fills” are the *same computation*.

Why we use it here. Telling a planner *which* ingredient to expedite is the whole product, and the shadow price *is* that answer — earned from the data, not assumed. Relaxing the hard material-balance rules into priced terms also turns an awkward constrained problem into a smooth one a continuous optimizer can chew through quickly. A method that only spat out a plan (without the per-constraint marginal value) would hide the very number the tool exists to surface.

D.4 The Augmented Lagrangian (ALM) & ρ — the robust version

What it is. A plain Lagrangian can misbehave before the prices are learned: with only a linear price term, the inner problem can run away to a nonsense answer, and the prices can oscillate. The **Augmented Lagrangian Method** fixes this by adding one more term — a **quadratic penalty** $(\rho/2) \cdot (\text{violation})^2$ — on top of the linear price. Think of it as a **soft, bowl-shaped wall** that gently pushes back the moment you cross the constraint, keeping the inner problem well-behaved *while* the linear term still does the real job of earning the true shadow price. The weight ρ controls how firm that soft wall is, and it doubles as the **step size** for updating the prices: after each round it nudges each λ by $\rho \times$ its violation, and ρ itself grows if progress stalls (capped, so a genuine shortage doesn’t send it to infinity).

Why we use it here. Every shortage plan is, strictly, *infeasible* — you cannot fill every order — and a plain Lagrangian is fragile exactly there. The quadratic augmentation makes convergence stable on precisely the problems we care about, and using ρ as the step size means there is **no separate learning-rate to hand-tune** (a knob that could only be set wrong). It is the difference between a method that works in the demo and one that works every morning on real, over-committed demand.

D.5 L-BFGS-B — the inner quantity solver

What it is. Inside each ALM round is a sub-problem: with the prices held fixed, find the **production quantities** that minimize the augmented objective — a smooth continuous problem with simple limits (you can make between 0 and the order quantity of each thing). **L-BFGS-B** is the standard solver for exactly this shape. Unpacking the name: **BFGS** is a *quasi-Newton* method — it heads roughly toward the bottom using not just the slope (gradient) but an estimate of the *curvature*, which lets it take much smarter steps than plain gradient descent; **L** (“limited-memory”) means it *approximates* that curvature from just the last few steps instead of storing a huge matrix, so it scales; and **-B** means it respects *bounds* (the 0-to-order-quantity box). Plainly: a fast, low-memory hill-descender that knows the shape of the hill and stays inside the fence.

Why we use it here. The primal sub-problem is smooth, bounded, and of moderate size — the textbook case L-BFGS-B is built for. It converges quickly without a manual step size or a full curvature matrix, so each of the many ALM rounds stays cheap. A simpler gradient descent would need careful step-size tuning and take far more iterations; a full Newton method would choke on the matrix. L-BFGS-B is the right middle.

D.6 Largest-remainder apportionment — placing whole batches

What it is. The prices tell us each order *deserves*, say, 67% and 33% of a shared pool — but production comes in indivisible **whole batches**, so you can't hand out a fraction of one. Largest-remainder apportionment is the classic way to split a fixed number of indivisible things proportionally: give everyone the *whole number* part of their fair share first, then hand the leftover units, one at a time, to whoever has the **largest fractional remainder** left over. It is the very method used to apportion seats in a legislature to states by population — same problem, same fix.

Why we use it here. It is **conservation-exact by construction** — the pieces always add up to exactly the pool, never more (no double-spending) and never less (no stranding) — while staying as close to the λ -proportional split as whole batches allow. Simply rounding each order's fractional share independently would break that: two orders could each round *up* off the same pool and conjure material that isn't there. When one batch is large relative to the pool, indivisibility forces the whole batch onto one order and the other becomes the visible

competed loser — an honest outcome the graph draws as such.

D.7 The costate shortcut (Pontryagin) — daily prices without a daily re-solve

What it is. This one is an *optional refinement*, not part of the core pipeline. In optimal-control theory (Pontryagin's maximum principle) every constraint's price has a companion quantity called a **costate** — think of it as the shadow price carried *forward through time*, evolving as conditions change day to day. The useful consequence: to update that price for a new day, you do *not* have to re-solve the whole problem; you can **propagate it locally** — nudge it in the direction of that day's constraint tension. The same idea shows up in the Augmented Lagrangian as its **dual step** ($\lambda \leftarrow \lambda + \rho \cdot g$): a single step re-prices λ from the current violation without touching the quantities.

Why we use it here. Prices are solved weekly to keep the run fast, but a shortage really bites on a *particular day*. Genuinely re-solving the Lagrangian at daily resolution would multiply the runtime; the costate shortcut instead takes **one dual step per day** from that day's supply-vs-demand, *simulating* what a daily re-solve would do to λ at almost no cost (Appendix C.8 has the full reasoning; it is the `FROBOZZ_DAILY_LAMBDA` option). It is a first-order, predictive correction — exactly the right ambition for a price that is itself an estimate of the future.

The pipeline in one breath. QUBO writes the yes/no selection as a score; **simulated annealing** searches that score for a good seed; the **Augmented Lagrangian** relaxes the hard material-balance rules into priced terms and, with **L-BFGS-B** solving the quantities inside each round, *earns* the shadow prices λ that say which constraint costs the most; **largest-remainder apportionment** settles those prices into real whole batches without inventing or losing a pound; and, when you want it, the **costate dual step** re-prices λ day by day without a re-solve. Each tool is standard and each is matched to the exact shape of its stage — combinatorial, continuous, then discrete.